



Step two: make a Remote implementation

① Implement the Remote interface

Your service has to implement the remote interface—the one with the methods your client is going to call.

```
public class MyRemoteImpl extends UnicastRemoteObject implements MyRemote {
    public String sayHello() {
        return "Server says, 'Hey'";
    }
    // more code in class
}
```

The compiler will make sure that you've implemented all the methods from the interface you implement. In this case, there's only one.

② Extend UnicastRemoteObject

In order to work as a remote service object, your object needs some functionality related to 'being remote'. The simplest way is to extend `UnicastRemoteObject` (from the `java.rmi.server` package) and let that class (your superclass) do the work for you.

```
public class MyRemoteImpl extends UnicastRemoteObject implements MyRemote {
```

③ Write a no-arg constructor that declares a RemoteException

Your new superclass, `UnicastRemoteObject`, has one little problem—its constructor throws a `RemoteException`. The only way to deal with this is to declare a constructor for your remote implementation, just so that you have a place to declare the `RemoteException`. Remember, when a class is instantiated, its superclass constructor is always called. If your superclass constructor throws an exception, you have no choice but to declare that your constructor also throws an exception.

```
public MyRemoteImpl() throws RemoteException { }
```

You don't have to put anything in the constructor. You just need a way to declare that your superclass constructor throws an exception.

④ Register the service with the RMI registry

Now that you've got a remote service, you have to make it available to remote clients. You do this by instantiating it and putting it into the RMI registry (which must be running or this line of code fails). When you register the implementation object, the RMI system actually puts the *stub* in the registry, since that's what the client really needs. Register your service using the static `rebind()` method of the `java.rmi.Naming` class.

```
try {
    MyRemote service = new MyRemoteImpl();
    Naming.rebind("RemoteHello", service);
} catch (Exception ex) { ... }
```

Give your service a name (that clients can use to look it up in the registry) and register it with the RMI registry. When you bind the service object, RMI swaps the service for the stub and puts the stub in the registry.

Step three: generate stubs and skeletons

① Run `rmic` on the remote implementation class (not the remote interface)

The `rmic` tool, which comes with the Java software development kit, takes a service implementation and creates two new classes, the stub and the skeleton. It uses a naming convention that is the name of your remote implementation, with either `_Stub` or `_Skel` added to the end. There are other options with `rmic`, including not generating skeletons, seeing what the source code for these classes looked like, and even using IIOP as the protocol. The way we're doing it here is the way you'll usually do it. The classes will land in the current directory (i.e. whatever you did a `cd` to). Remember, `rmic` must be able to see your implementation class, so you'll probably run `rmic` from the directory where your remote implementation is located. (We're deliberately not using packages here, to make it simpler. In the Real World, you'll need to account for package directory structures and fully-qualified names).

Notice that you don't say ".class" on the end. Just the class name.

```
File Edit Window Help Whuffie
%rmic MyRemoteImpl
```

RMIC generates two new classes for the helper objects.



MyRemoteImpl_Stub.class



MyRemoteImpl_Skel.class

Step four: run `rmiregistry`

① Bring up a terminal and start the `rmiregistry`.

Be sure you start it from a directory that has access to your classes. The simplest way is to start it from your 'classes' directory.

```
File Edit Window Help Huh?
%rmiregistry
```

Step five: start the service

① Bring up another terminal and start your service

This might be from a `main()` method in your remote implementation class, or from a separate launcher class. In this simple example, we put the starter code in the implementation class, in a `main` method that instantiates the object and registers it with RMI registry.

```
File Edit Window Help Huh?
%java MyRemoteImpl
```

Complete code for the server side



The Remote interface:

```
import java.rmi.*;
```

RemoteException and Remote interface are in java.rmi package.

```
public interface MyRemote extends Remote {
```

Your interface MUST extend java.rmi.Remote

```
    public String sayHello() throws RemoteException;
```

All of your remote methods must declare a RemoteException.

```
}
```

The Remote service (the implementation):

```
import java.rmi.*;
```

UnicastRemoteObject is in the java.rmi.server package.

```
import java.rmi.server.*;
```

Extending UnicastRemoteObject is the easiest way to make a remote object.

```
public class MyRemoteImpl extends UnicastRemoteObject implements MyRemote {
```

You MUST implement your remote interface!!

```
    public String sayHello() {
```

You have to implement all the interface methods, of course. But notice that you do NOT have to declare the RemoteException.

```
        return "Server says, 'Hey'";
```

```
    }
```

```
    public MyRemoteImpl() throws RemoteException { }
```

Your superclass constructor (for UnicastRemoteObject) declares an exception, so YOU must write a constructor, because it means that your constructor is calling risky code (its super constructor).

```
    public static void main (String[] args) {
```

Make the remote object, then 'bind' it to the rmiregistry using the static Naming.rebind(). The name you register it under is the name clients will use to look it up in the RMI registry.

```
        try {
```

```
            MyRemote service = new MyRemoteImpl();
```

```
            Naming.rebind("RemoteHello", service);
```

```
        } catch (Exception ex) {
```

```
            ex.printStackTrace();
```

```
        }
```

```
    }
```

```
}
```

How does the client get the stub object?

The client has to get the stub object (our proxy), since that's the thing the client will call methods on. And that's where the RMI registry comes in. The client does a 'lookup', like going to the white pages of a phone book, and essentially says, "Here's a name, and I'd like the stub that goes with that name."

Let's take a look at the code we need to lookup and retrieve a stub object.

Here's how it works.



Code Up Close

The client always uses the remote interface as the type of the service. In fact, the client never needs to know the actual class name of your remote service.

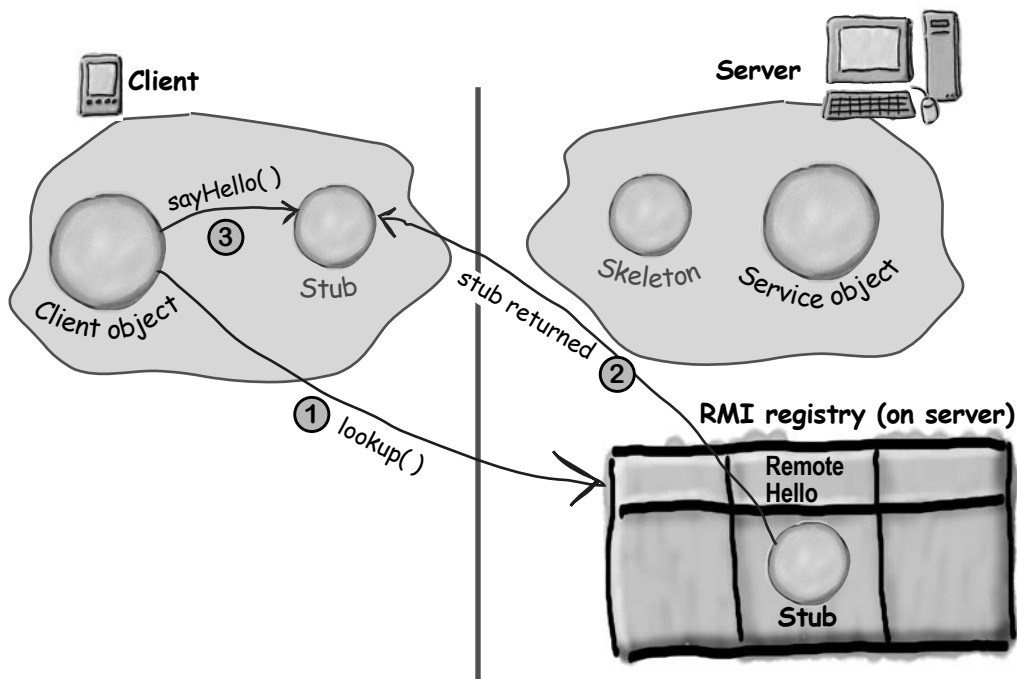
```
MyRemote service =  
(MyRemote) Naming.lookup("rmi://127.0.0.1/RemoteHello");
```

You have to cast it to the interface, since the lookup method returns type Object.

lookup() is a static method of the Naming class.

This must be the name that the service was registered under.

The host name or IP address where the service is running.



How it works...

① Client does a lookup on the RMI registry

```
Naming.lookup("rmi://127.0.0.1/RemoteHello");
```

② RMI registry returns the stub object

(as the return value of the lookup method) and RMI deserializes the stub automatically. You **MUST** have the stub class (that rmic generated for you) on the client or the stub won't be deserialized.

③ Client invokes a method on the stub, as if the stub IS the real service

Complete client code

```
import java.rmi.*;

public class MyRemoteClient {
    public static void main (String[] args) {
        new MyRemoteClient().go();
    }

    public void go() {
        try {
            MyRemote service = (MyRemote) Naming.lookup("rmi://127.0.0.1/RemoteHello");

            String s = service.sayHello();

            System.out.println(s);
        } catch (Exception ex) {
            ex.printStackTrace();
        }
    }
}
```

The Naming class (for doing the rmi registry lookup) is in the java.rmi package.

It comes out of the registry as type Object, so don't forget the cast.

You need the IP address or hostname.

and the name used to bind/rebind the service.

It looks just like a regular old method call! (Except it must acknowledge the RemoteException.)



Geek Bits

How does the client get the stub class?

Now we get to the interesting question. Somehow, some way, the client must have the stub class (that you generated earlier using `rmic`) at the time the client does the lookup, or else the stub won't be deserialized on the client and the whole thing blows up. The client also needs classes for any serialized objects returned by method calls to the remote object. In a simple system, you can simply hand-deliver these classes to the client.

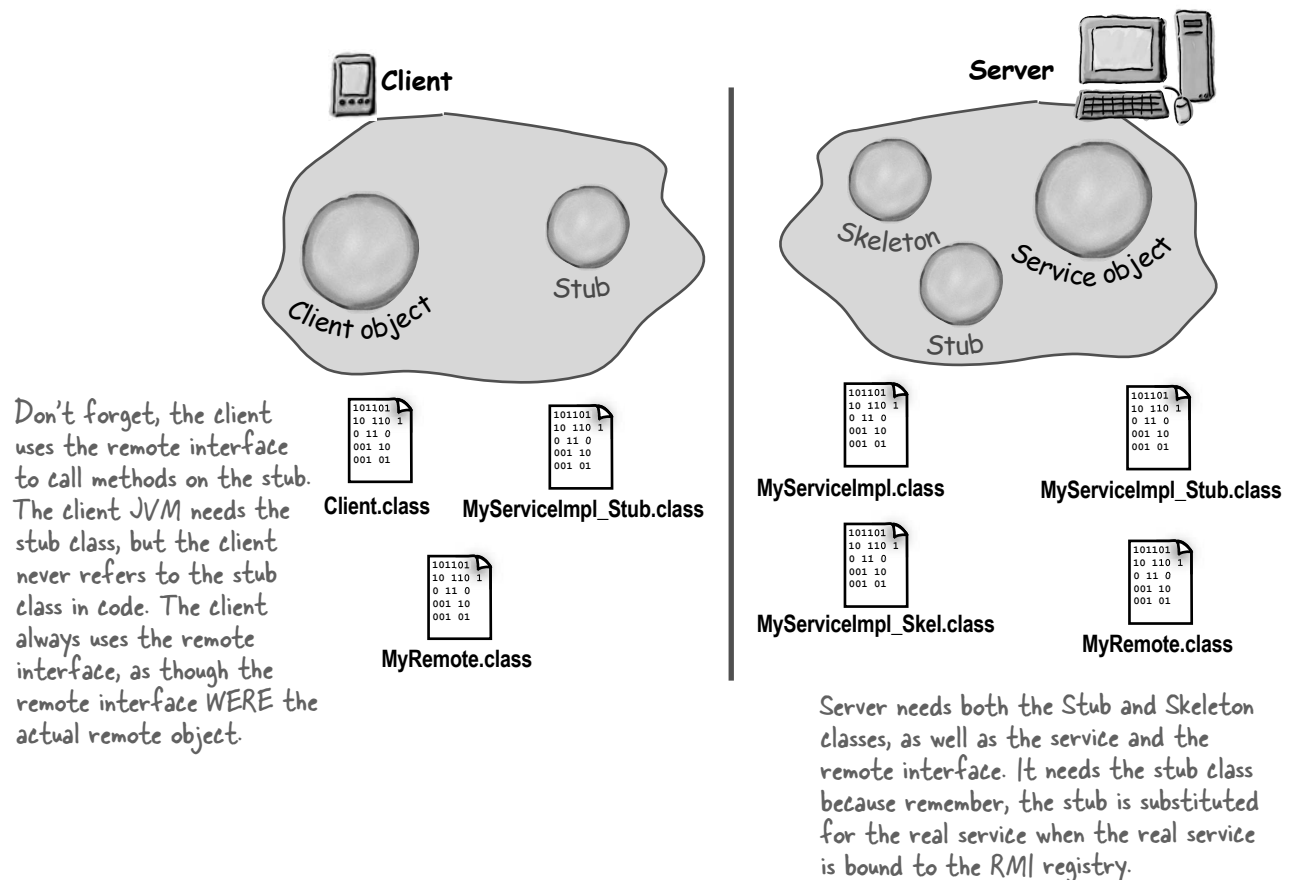
There's a much cooler way, although it's beyond the scope of this book. But just in case you're interested, the cooler way is called "dynamic class downloading". With dynamic class downloading, Serialized objects (like the stub) are "stamped" with a URL that tells the RMI system on the client where to find the class file for that object. Then, in the process of deserializing an object, if RMI can't find the class locally, it uses that URL to do an HTTP Get to retrieve the class file. So you'd need a simple web server to serve up class files, and you'd also need to change some security parameters on the client. There are a few other tricky issues with dynamic class downloading, but that's the overview.

For the stub object specifically, there's *another* way the client can get the class. This is only available in Java 5, though. We'll briefly talk about this near the end of the chapter.



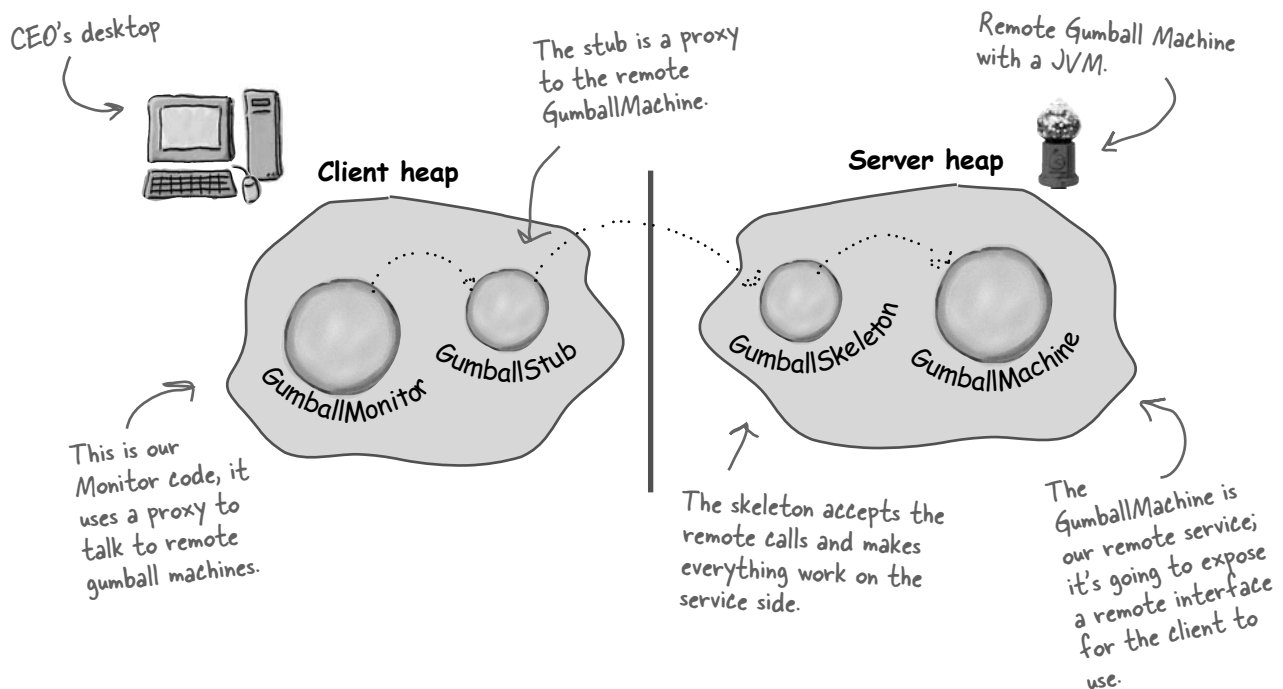
The top three things programmers do wrong with RMI are:

- 1) Forget to start rmiregistry before starting remote service (when the service is registered using Naming.rebind(), the rmiregistry must be running!)
- 2) Forget to make arguments and return types serializable (you won't know until runtime; this is not something the compiler will detect.)
- 3) Forget to give the stub class to the client.



Back to our GumballMachine remote proxy

Okay, now that you have the RMI basics down, you've got the tools you need to implement the gumball machine remote proxy. Let's take a look at how the GumballMachine fits into this framework:



Getting the GumballMachine ready to be a remote service

The first step in converting our code to use the remote proxy is to enable the GumballMachine to service remote requests from clients. In other words, we're going to make it into a service. To do that, we need to:

- 1) Create a remote interface for the GumballMachine. This will provide a set of methods that can be called remotely.
- 2) Make sure all the return types in the interface are serializable.
- 3) Implement the interface in a concrete class.

We'll start with the remote interface:

Don't forget to import java.rmi.*

```
import java.rmi.*;
```

This is the remote interface.

```
public interface GumballMachineRemote extends Remote {
    public int getCount() throws RemoteException;
    public String getLocation() throws RemoteException;
    public State getState() throws RemoteException;
}
```

All return types need to be primitive or Serializable...

Here are the methods we're going to support. Each one throws RemoteException.

We have one return type that isn't Serializable: the State class. Let's fix it up...

```
import java.io.*;
```

Serializable is in the java.io package.

```
public interface State extends Serializable {
    public void insertQuarter();
    public void ejectQuarter();
    public void turnCrank();
    public void dispense();
}
```

Then we just extend the Serializable interface (which has no methods in it). And now State in all the subclasses can be transferred over the network.

remote interface for the gumball machine

Actually, we're not done with Serializable yet; we have one problem with State. As you may remember, each State object maintains a reference to a gumball machine so that it can call the gumball machine's methods and change its state. We don't want the entire gumball machine serialized and transferred with the State object. There is an easy way to fix this:

```
public class NoQuarterState implements State {  
    transient GumballMachine gumballMachine;  
  
    // all other methods here  
}
```

In each implementation of State, we add the transient keyword to the GumballMachine instance variable. This tells the JVM not to serialize this field. Note that this can be slightly dangerous if you try to access this field once it's been serialized and transferred.

We've already implemented our GumballMachine, but we need to make sure it can act as a service and handle requests coming from over the network. To do that, we have to make sure the GumballMachine is doing everything it needs to implement the GumballMachineRemote interface.

As you've already seen in the RMI detour, this is quite simple, all we need to do is add a couple of things...

First, we need to import the rmi packages.

```
import java.rmi.*;  
import java.rmi.server.*;
```

GumballMachine is going to subclass the UnicastRemoteObject; this gives it the ability to act as a remote service.

GumballMachine also needs to implement the remote interface...

```
public class GumballMachine  
    extends UnicastRemoteObject implements GumballMachineRemote  
{  
    // instance variables here  
  
    public GumballMachine(String location, int numberGumballs) throws RemoteException {  
        // code here  
    }  
  
    public int getCount() {  
        return count;  
    }  
  
    public State getState() {  
        return state;  
    }  
  
    public String getLocation() {  
        return location;  
    }  
  
    // other methods here  
}
```

...and the constructor needs to throw a remote exception, because the superclass does.

That's it! Nothing changes here at all!

Registering with the RMI registry...

That completes the gumball machine service. Now we just need to fire it up so it can receive requests. First, we need to make sure we register it with the RMI registry so that clients can locate it.

We're going to add a little code to the test drive that will take care of this for us:

```
public class GumballMachineTestDrive {

    public static void main(String[] args) {
        GumballMachineRemote gumballMachine = null;
        int count;
        if (args.length < 2) {
            System.out.println("GumballMachine <name> <inventory>");
            System.exit(1);
        }

        try {
            count = Integer.parseInt(args[1]);

            gumballMachine =
                new GumballMachine(args[0], count);
            Naming.rebind("//" + args[0] + "/gumballmachine", gumballMachine);
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

First we need to add a try/catch block around the gumball instantiation because our constructor can now throw exceptions.

We also add the call to Naming.rebind, which publishes the GumballMachine stub under the name gumballmachine.

Let's go ahead and get this running...

Run this first.

This gets the RMI registry service up and running.

We're using the "official" Mighty Gumball machines, you should substitute your own machine name here.

The image shows two overlapping terminal windows. The top window has a title bar 'File Edit Window Help Huh?' and contains the command `% rmiregistry`. The bottom window also has a title bar 'File Edit Window Help Huh?' and contains the command `% java GumballMachineTestDrive seattle.mightygumball.com 100`. Arrows point from the text annotations to these commands.

Run this second.

This gets the GumballMachine up and running and registers it with the RMI registry.

Now for the GumballMonitor client...

Remember the GumballMonitor? We wanted to reuse it without having to rewrite it to work over a network. Well, we're pretty much going to do that, but we do need to make a few changes.

```
import java.rmi.*;

public class GumballMonitor {
    GumballMachineRemote machine;

    public GumballMonitor(GumballMachineRemote machine) {
        this.machine = machine;
    }

    public void report() {
        try {
            System.out.println("Gumball Machine: " + machine.getLocation());
            System.out.println("Current inventory: " + machine.getCount() + " gumballs");
            System.out.println("Current state: " + machine.getState());
        } catch (RemoteException e) {
            e.printStackTrace();
        }
    }
}
```

We need to import the RMI package because we are using the RemoteException class below...

Now we're going to rely on the remote interface rather than the concrete GumballMachine class.

We also need to catch any remote exceptions that might happen as we try to invoke methods that are ultimately happening over the network.



Writing the Monitor test drive

Now we've got all the pieces we need. We just need to write some code so the CEO can monitor a bunch of gumball machines:

Here's the monitor test drive. The CEO is going to run this!

```
import java.rmi.*;

public class GumballMonitorTestDrive {

    public static void main(String[] args) {
        String[] location = {"rmi://santafe.mightygumball.com/gumballmachine",
                             "rmi://boulder.mightygumball.com/gumballmachine",
                             "rmi://seattle.mightygumball.com/gumballmachine"};

        GumballMonitor[] monitor = new GumballMonitor[location.length];

        for (int i=0; i < location.length; i++) {
            try {
                GumballMachineRemote machine =
                    (GumballMachineRemote) Naming.lookup(location[i]);
                monitor[i] = new GumballMonitor(machine);
                System.out.println(monitor[i]);
            } catch (Exception e) {
                e.printStackTrace();
            }
        }

        for(int i=0; i < monitor.length; i++) {
            monitor[i].report();
        }
    }
}
```

Here's all the locations we're going to monitor.

We create an array of locations, one for each machine.

We also create an array of monitors.

Now we need to get a proxy to each remote machine.

Then we iterate through each machine and print out its report.



Code Up Close

This returns a proxy to the remote Gumball Machine (or throws an exception if one can't be located).

Remember, `Naming.lookup()` is a static method in the `RMI` package that takes a location and service name and looks it up in the `rmiregistry` at that location.

```
try {
    GumballMachineRemote machine =
        (GumballMachineRemote) Naming.lookup(location[i]);

    monitor[i] = new GumballMonitor(machine);

} catch (Exception e) {
    e.printStackTrace();
}
```

Once we get a proxy to the remote machine, we create a new `GumballMonitor` and pass it the machine to monitor.

Another demo for the CEO of Mighty Gumball...

Okay, it's time to put all this work together and give another demo. First let's make sure a few gumball machines are running the new code:

On each machine, run `rmiregistry` in the background or from a separate terminal window...

...and then run the `GumballMachine`, giving it a location and an initial gumball count.

```
File Edit Window Help Huh?
% rmiregistry &
% java GumballMachine santafe.mightygumball.com 100

File Edit Window Help Huh?
% rmiregistry &
% java GumballMachine boulder.mightygumball.com 100

File Edit Window Help Huh?
% rmiregistry &
% java GumballMachine seattle.mightygumball.com 250
popular machine! ↗
```

And now let's put the monitor in the hands of the CEO.
Hopefully this time he'll love it:

```
File Edit Window Help GumballsAndBeyond
% java GumballMonitor
Gumball Machine: santafe.mightygumball.com
Current inventory: 99 gumballs
Current state: waiting for quarter

Gumball Machine: boulder.mightygumball.com
Current inventory: 44 gumballs
Current state: waiting for turn of crank

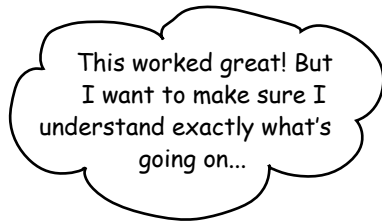
Gumball Machine: seattle.mightygumball.com
Current inventory: 187 gumballs
Current state: waiting for quarter
%
```

The monitor iterates over each remote machine and calls its getLocation(), getCount() and getState() methods.

This is amazing; it's going to revolutionize my business and blow away the competition!



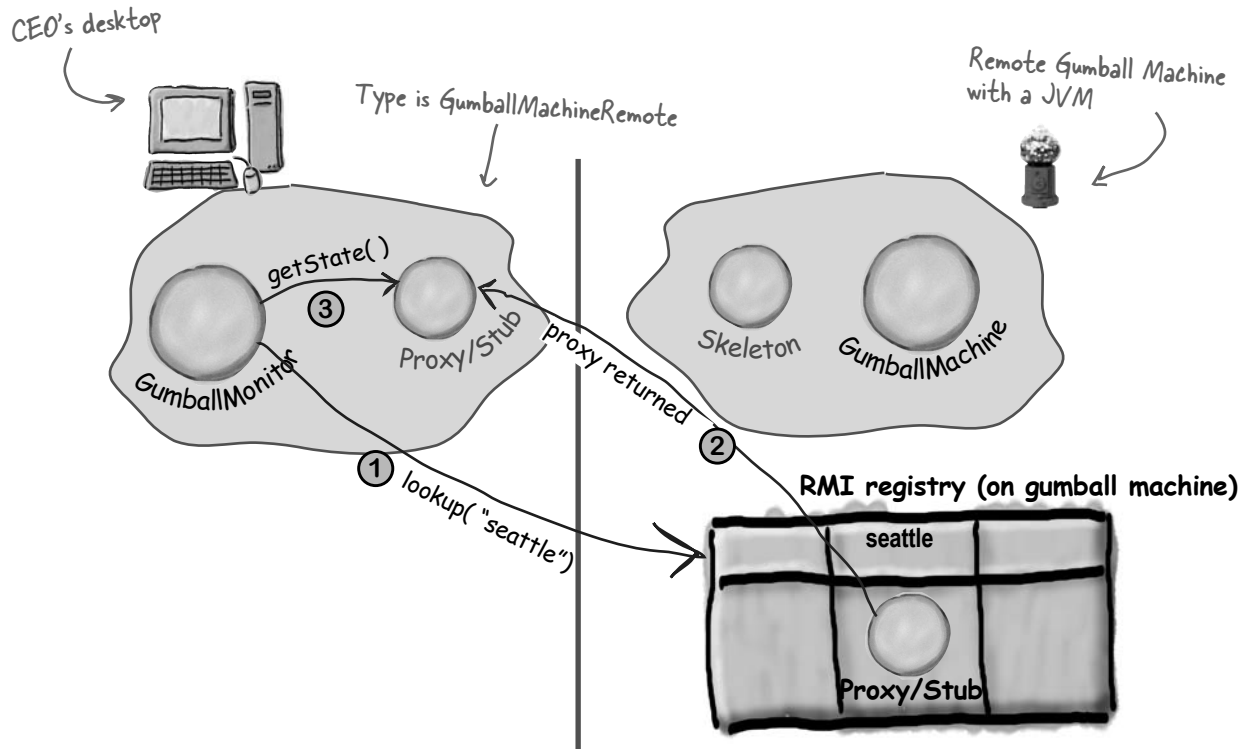
By invoking methods on the proxy, a remote call is made across the wire and a String, an integer and a State object are returned. Because we are using a proxy, the GumballMonitor doesn't know, or care, that calls are remote (other than having to worry about remote exceptions).



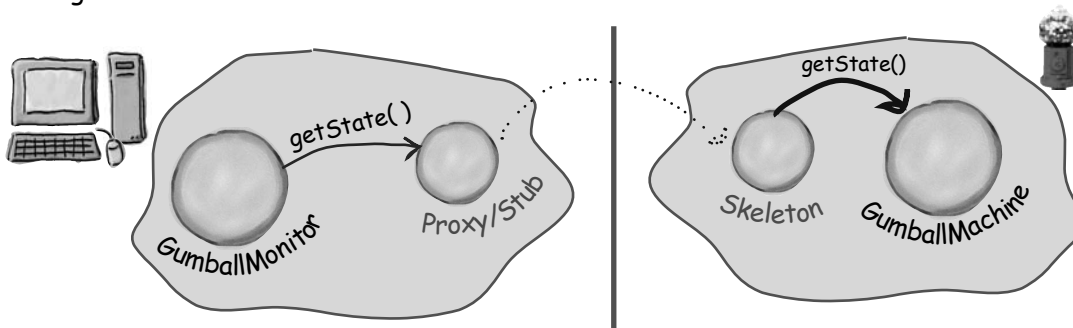
Behind the Scenes



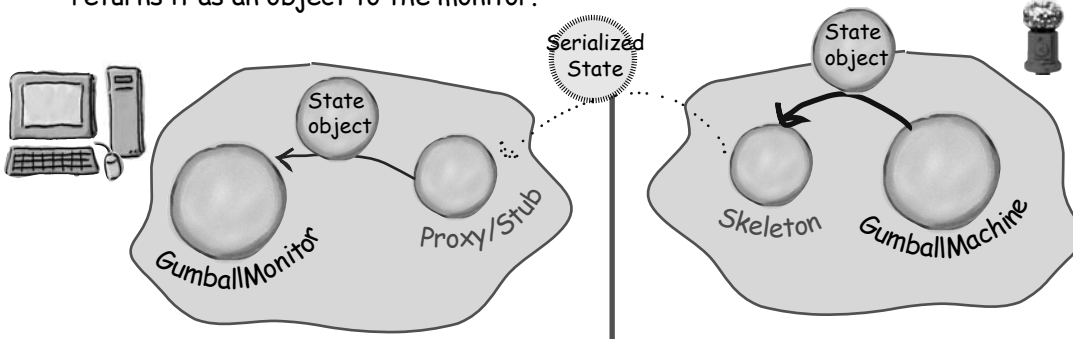
- 1 The CEO runs the monitor, which first grabs the proxies to the remote gumball machines and then calls `getState()` on each one (along with `getCount()` and `getLocation()`).



- 2 `getState()` is called on the proxy, which forwards the call to the remote service. The skeleton receives the request and then forwards it to the gumball machine.



- 3 GumballMachine returns the state to the skeleton, which serializes it and transfers it back over the wire to the proxy. The proxy deserializes it and returns it as an object to the monitor.



The monitor hasn't changed at all, except it knows it may encounter remote exceptions. It also uses the `GumballMachineRemote` interface rather than a concrete implementation.

Likewise, the `GumballMachine` implements another interface and may throw a remote exception in its constructor, but other than that, the code hasn't changed.

We also have a small bit of code to register and locate stubs using the RMI registry. But no matter what, if we were writing something to work over the Internet, we'd need some kind of locator service.

The Proxy Pattern defined

We've already put a lot of pages behind us in this chapter; as you can see, explaining the Remote Proxy is quite involved. Despite that, you'll see that the definition and class diagram for the Proxy Pattern is actually fairly straightforward. Note that Remote Proxy is one implementation of the general Proxy Pattern; there are actually quite a few variations of the pattern, and we'll talk about them later. For now, let's get the details of the general pattern down.

Here's the Proxy Pattern definition:

The Proxy Pattern provides a surrogate or placeholder for another object to control access to it.

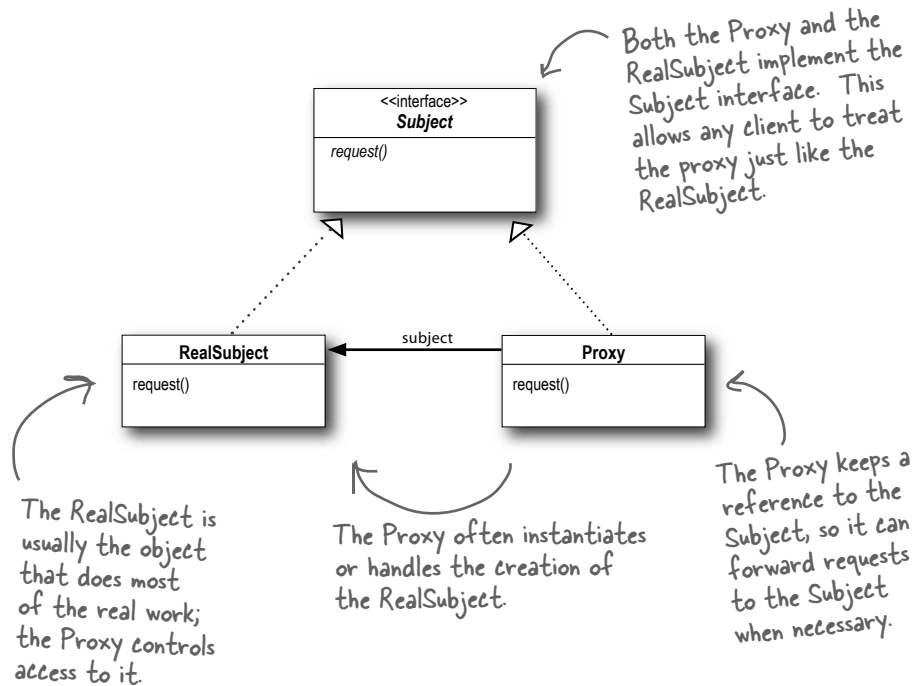
Well, we've seen how the Proxy Pattern provides a surrogate or placeholder for another object. We've also described the proxy as a "representative" for another object.

But what about a proxy controlling access? That sounds a little strange. No worries. In the case of the gumball machine, just think of the proxy controlling access to the remote object. The proxy needed to control access because our client, the monitor, didn't know how to talk to a remote object. So in some sense the remote proxy controlled access so that it could handle the network details for us. As we just discussed, there are many variations of the Proxy Pattern, and the variations typically revolve around the way the proxy "controls access." We're going to talk more about this later, but for now here are a few ways proxies control access:

- As we know, a remote proxy controls access to a remote object.
- A virtual proxy controls access to a resource that is expensive to create.
- A protection proxy controls access to a resource based on access rights.

Now that you've got the gist of the general pattern, check out the class diagram...

Use the Proxy Pattern to create a representative object that controls access to another object, which may be remote, expensive to create or in need of securing.



Let's step through the diagram...

First we have a Subject, which provides an interface for the RealSubject and the Proxy. By implementing the same interface, the Proxy can be substituted for the RealSubject anywhere it occurs.

The RealSubject is the object that does the real work. It's the object that the Proxy represents and controls access to.

The Proxy holds a reference to the RealSubject. In some cases, the Proxy may be responsible for creating and destroying the RealSubject. Clients interact with the RealSubject through the Proxy. Because the Proxy and RealSubject implement the same interface (Subject), the Proxy can be substituted anywhere the subject can be used. The Proxy also controls access to the RealSubject; this control may be needed if the Subject is running on a remote machine, if the Subject is expensive to create in some way or if access to the subject needs to be protected in some way.

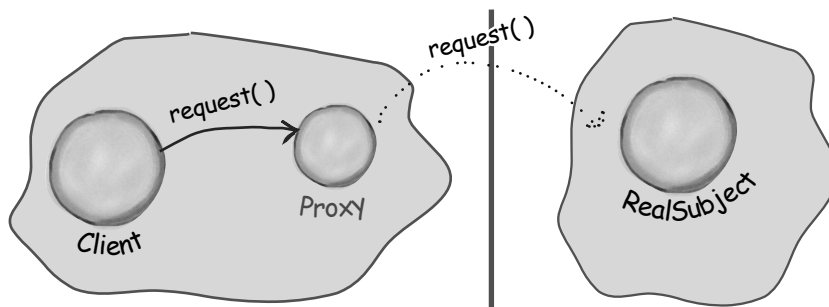
Now that you understand the general pattern, let's look at some other ways of using proxy beyond the Remote Proxy...

Get ready for Virtual Proxy

Okay, so far you've seen the definition of the Proxy Pattern and you've taken a look at one specific example: the *Remote Proxy*. Now we're going to take a look at a different type of proxy, the *Virtual Proxy*. As you'll discover, the Proxy Pattern can manifest itself in many forms, yet all the forms follow roughly the general proxy design. Why so many forms? Because the proxy pattern can be applied to a lot of different use cases. Let's check out the Virtual Proxy and compare it to Remote Proxy:

Remote Proxy

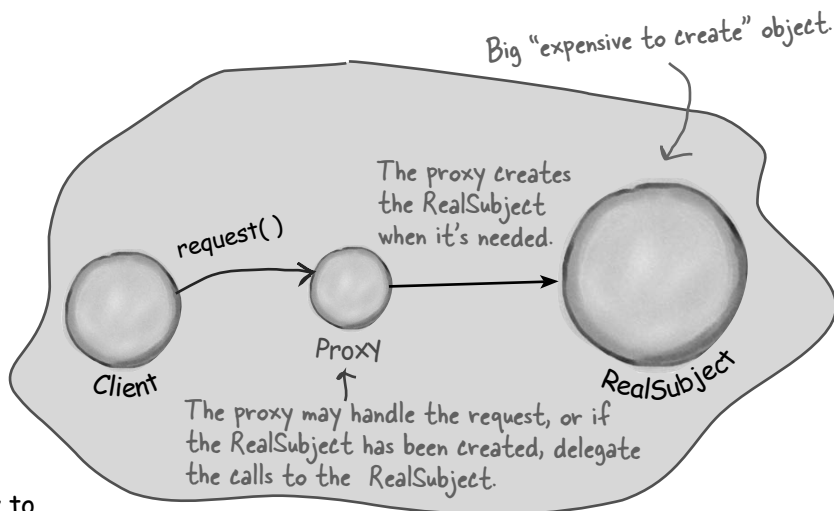
With Remote Proxy, the proxy acts as a local representative for an object that lives in a different JVM. A method call on the proxy results in the call being transferred over the wire, invoked remotely, and the result being returned back to the proxy and then to the Client.



We know this diagram pretty well by now...

Virtual Proxy

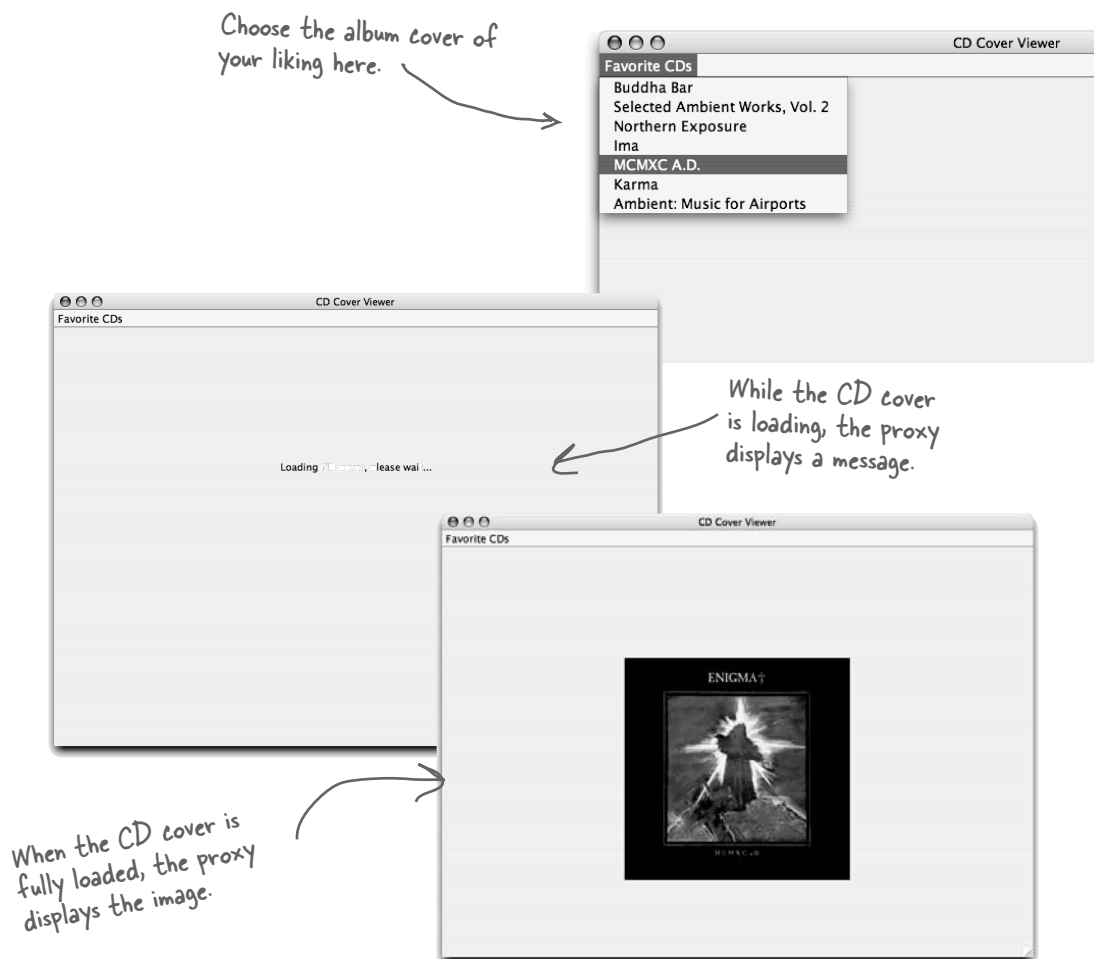
Virtual Proxy acts as a representative for an object that may be expensive to create. The Virtual Proxy often defers the creation of the object until it is needed; the Virtual Proxy also acts as a surrogate for the object before and while it is being created. After that, the proxy delegates requests directly to the RealSubject.



Displaying CD covers

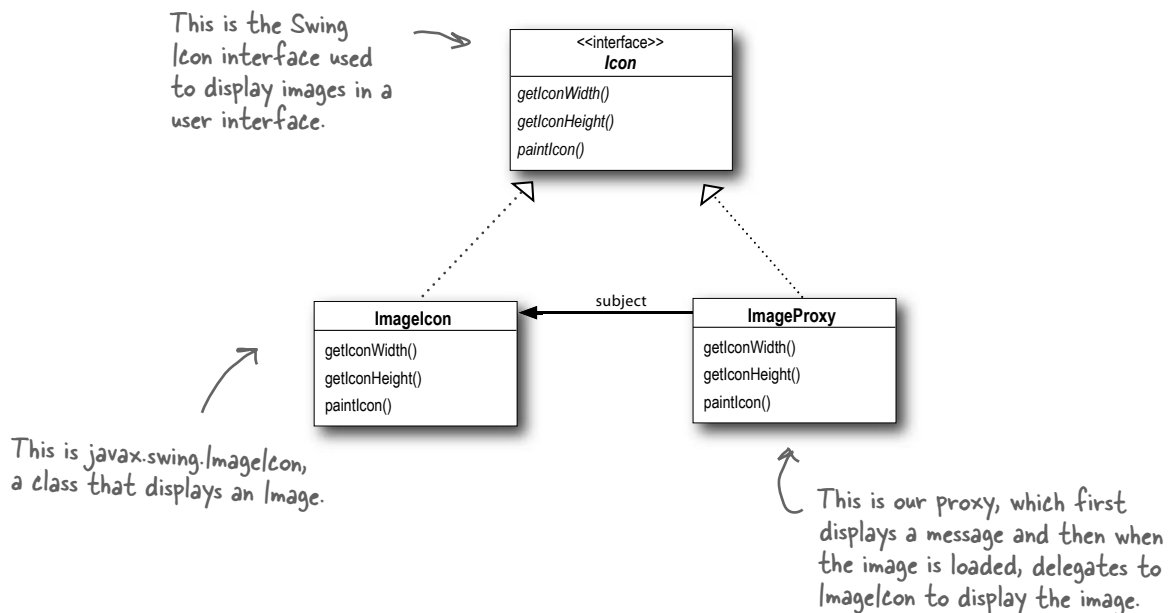
Let's say you want to write an application that displays your favorite compact disc covers. You might create a menu of the CD titles and then retrieve the images from an online service like Amazon.com. If you're using Swing, you might create an Icon and ask it to load the image from the network. The only problem is, depending on the network load and the bandwidth of your connection, retrieving a CD cover might take a little time, so your application should display something while you are waiting for the image to load. We also don't want to hang up the entire application while it's waiting on the image. Once the image is loaded, the message should go away and you should see the image.

An easy way to achieve this is through a virtual proxy. The virtual proxy can stand in place of the icon, manage the background loading, and before the image is fully retrieved from the network, display "Loading CD cover, please wait...". Once the image is loaded, the proxy delegates the display to the Icon.



Designing the CD cover Virtual Proxy

Before writing the code for the CD Cover Viewer, let's look at the class diagram. You'll see this looks just like our Remote Proxy class diagram, but here the proxy is used to hide an object that is expensive to create (because we need to retrieve the data for the Icon over the network) as opposed to an object that actually lives somewhere else on the network.



How ImageProxy is going to work:

- ❶ **ImageProxy first creates an ImageIcon and starts loading it from a network URL.**
- ❷ **While the bytes of the image are being retrieved, ImageProxy displays “Loading CD cover, please wait...”.**
- ❸ **When the image is fully loaded, ImageProxy delegates all method calls to the image icon, including paintIcon(), getWidth() and getHeight().**
- ❹ **If the user requests a new image, we'll create a new proxy and start the process over.**

Writing the Image Proxy

```

class ImageProxy implements Icon {
    ImageIcon imageIcon;
    URL imageURL;
    Thread retrievalThread;
    boolean retrieving = false;

    public ImageProxy(URL url) { imageURL = url; }

    public int getIconWidth() {
        if (imageIcon != null) {
            return imageIcon.getIconWidth();
        } else {
            return 800;
        }
    }

    public int getIconHeight() {
        if (imageIcon != null) {
            return imageIcon.getIconHeight();
        } else {
            return 600;
        }
    }

    public void paintIcon(final Component c, Graphics g, int x, int y) {
        if (imageIcon != null) {
            imageIcon.paintIcon(c, g, x, y);
        } else {
            g.drawString("Loading CD cover, please wait...", x+300, y+190);
            if (!retrieving) {
                retrieving = true;
                retrievalThread = new Thread(new Runnable() {
                    public void run() {
                        try {
                            imageIcon = new ImageIcon(imageURL, "CD Cover");
                            c.repaint();
                        } catch (Exception e) {
                            e.printStackTrace();
                        }
                    }
                });
                retrievalThread.start();
            }
        }
    }
}

```

The ImageProxy implements the Icon interface.

<<interface>> Icon
getIconWidth() getIconHeight() paintIcon()

The ImageIcon is the REAL icon that we eventually want to display when it's loaded.

We pass the URL of the image into the constructor. This is the image we need to display once it's loaded!

We return a default width and height until the ImageIcon is loaded; then we turn it over to the ImageIcon.

Here's where things get interesting. This code paints the icon on the screen (by delegating to the ImageIcon). However, if we don't have a fully created ImageIcon, then we create one. Let's look at this closer on the next page...



Code Up Close

This method is called when it's time to paint the icon on the screen.

```
public void paintIcon(final Component c, Graphics g, int x, int y) {
    if (imageIcon != null) {
```

```
        ImageIcon.paintIcon(c, g, x, y);
```

← If we've got an icon already, we go ahead and tell it to paint itself.

```
    } else {
```

```
        g.drawString("Loading CD cover, please wait...", x+300, y+190);
```

```
        if (!retrieving) {
```

```
            retrieving = true;
```

```
            retrievalThread = new Thread(new Runnable() {
```

```
                public void run() {
```

```
                    try {
```

```
                        ImageIcon = new ImageIcon(imageURL, "CD Cover");
```

```
                        c.repaint();
```

```
                    } catch (Exception e) {
```

```
                        e.printStackTrace();
```

```
                    }
```

```
                }
```

```
            });
```

```
            retrievalThread.start();
```

```
        }
```

```
    }
```

```
}
```

← Otherwise we display the "loading" message.

Here's where we load the **REAL** icon image. Note that the image loading with `Icon/Image` is synchronous: the `ImageIcon` constructor doesn't return until the image is loaded. That doesn't give us much of a chance to do screen updates and have our message displayed, so we're going to do this asynchronously. See the "Code Way Up Close" on the next page for more...



Code Way Up Close

If we aren't already trying to retrieve the image...

```

if (!retrieving) {
    retrieving = true;

    retrievalThread = new Thread(new Runnable() {
        public void run() {
            try {
                ImageIcon = new ImageIcon(imageURL, "CD Cover");
                c.repaint();
            } catch (Exception e) {
                e.printStackTrace();
            }
        }
    });
    retrievalThread.start();
}

```

...then it's time to start retrieving it (in case you were wondering, only one thread calls paint, so we should be okay here in terms of thread safety).

We don't want to hang up the entire user interface, so we're going to use another thread to retrieve the image.

In our thread we instantiate the Icon object. Its constructor will not return until the image is loaded.

When we have the image, we tell Swing that we need to be repainted.

So, the next time the display is painted after the ImageIcon is instantiated, the paintIcon method will paint the image, not the loading message.

Design Puzzle

The ImageProxy class appears to have two states that are controlled by conditional statements. Can you think of another pattern that might clean up this code? How would you redesign ImageProxy?

```
class ImageProxy implements Icon {  
    // instance variables & constructor here
```

```
    public int getIconWidth() {  
        if (imageIcon != null) {  
            return imageIcon.getIconWidth();  
        } else {  
            return 800;  
        }  
    }
```

Two states

```
    public int getIconHeight() {  
        if (imageIcon != null) {  
            return imageIcon.getIconHeight();  
        } else {  
            return 600;  
        }  
    }
```

Two states

```
    public void paintIcon(final Component c, Graphics g, int x, int y) {  
        if (imageIcon != null) {  
            imageIcon.paintIcon(c, g, x, y);  
        } else {  
            g.drawString("Loading CD cover, please wait...", x+300, y+190);  
            // more code here  
        }  
    }  
}
```

Two states

Testing the CD Cover Viewer



Ready-bake
Code

Okay, it's time to test out this fancy new virtual proxy. Behind the scenes we've been baking up a new `ImageProxyTestDrive` that sets up the window, creates a frame, installs the menus and creates our proxy. We don't go through all that code in gory detail here, but you can always grab the source code and have a look, or check it out at the end of the chapter where we list all the source code for the Virtual Proxy.

Here's a partial view of the test drive code:

```
public class ImageProxyTestDrive {
    ImageComponent imageComponent;
    public static void main (String[] args) throws Exception {
        ImageProxyTestDrive testDrive = new ImageProxyTestDrive();
    }

    public ImageProxyTestDrive() throws Exception{

        // set up frame and menus

        Icon icon = new ImageProxy(initialURL);
        imageComponent = new ImageComponent(icon);
        frame.getContentPane().add(imageComponent);
    }
}
```

Here we create an image proxy and set it to an initial URL. Whenever you choose a selection from the CD menu, you'll get a new image proxy.

Next we wrap our proxy in a component so it can be added to the frame. The component will take care of the proxy's width, height and similar details.

Finally we add the proxy to the frame so it can be displayed.

Now let's run the test drive:

```
File Edit Window Help JustSomeOfTheCDsThatGotUsThroughThisBook
% java ImageProxyTestDrive
```

Running `ImageProxyTestDrive` should give you a window like this.

Things to try...

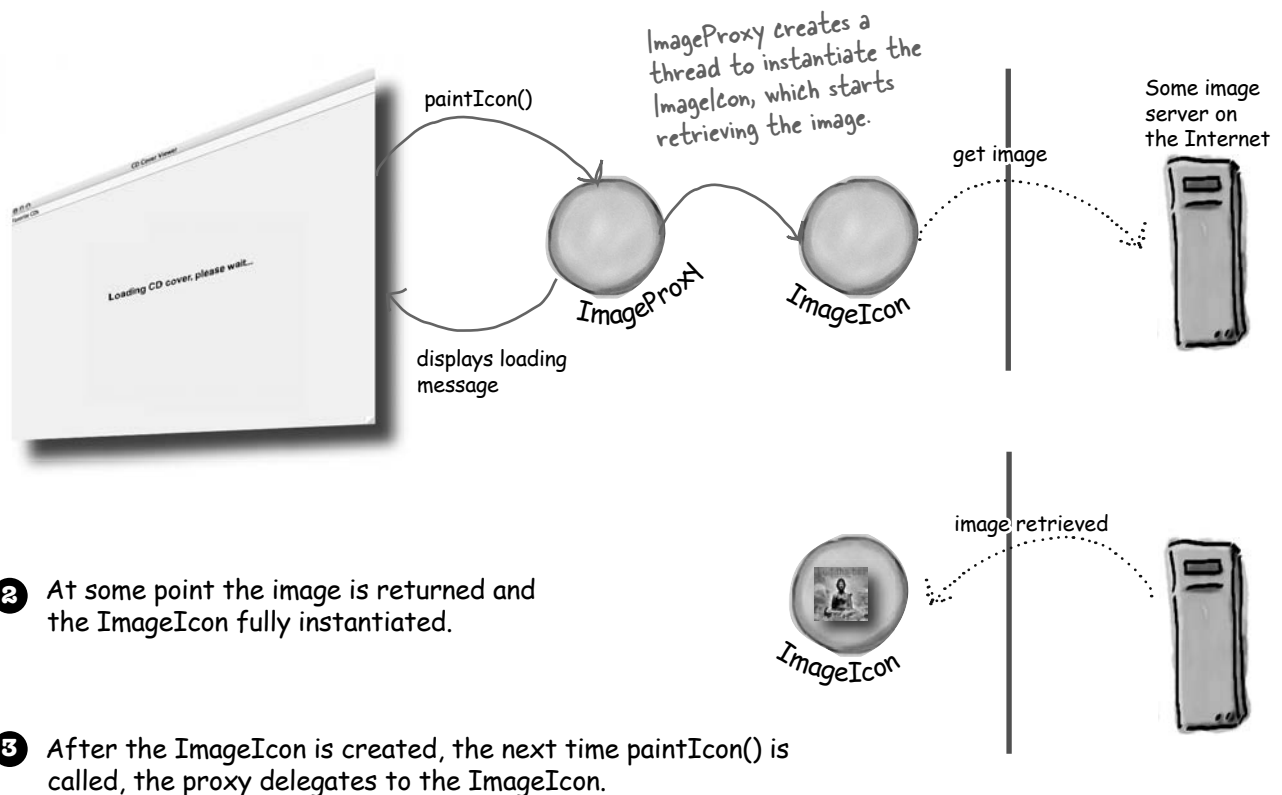
- ❶ Use the menu to load different CD covers; watch the proxy display "loading" until the image has arrived.
- ❷ Resize the window as the "loading" message is displayed. Notice that the proxy is handling the loading without hanging up the Swing window.
- ❸ Add your own favorite CDs to the `ImageProxyTestDrive`.



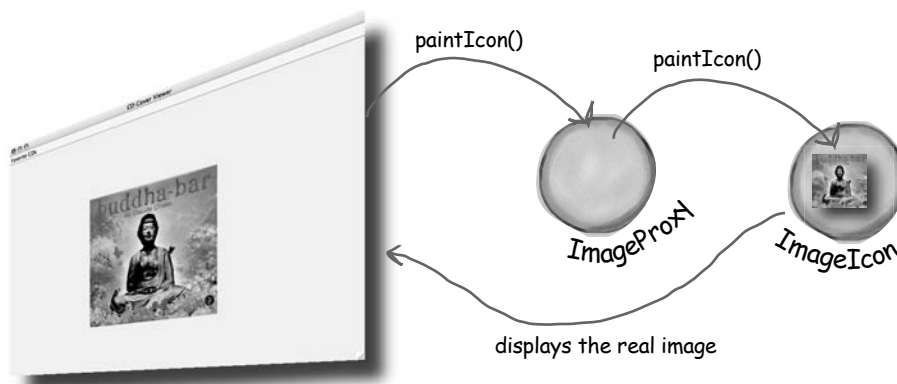
What did we do?

- 1 We created an ImageProxy for the display. The paintIcon() method is called and ImageProxy fires off a thread to retrieve the image and create the ImageIcon.

Behind the Scenes



- 2 At some point the image is returned and the ImageIcon fully instantiated.
- 3 After the ImageIcon is created, the next time paintIcon() is called, the proxy delegates to the ImageIcon.



there are no Dumb Questions

Q: The Remote Proxy and Virtual Proxy seem so different to me; are they really ONE pattern?

A: You'll find a lot of variants of the Proxy Pattern in the real world; what they all have in common is that they intercept a method invocation that the client is making on the subject. This level of indirection allows us to do many things, including dispatching requests to a remote subject, providing a representative for an expensive object as it is created, or, as you'll see, providing some level of protection that can determine which clients should be calling which methods. That's just the beginning; the general Proxy Pattern can be applied in many different ways, and we'll cover some of the other ways at the end of the chapter.

Q: ImageProxy seems just like a Decorator to me. I mean, we are basically wrapping one object with another and then delegating the calls to the Imagemcon. What am I missing?

A: Sometimes Proxy and Decorator look very similar, but their purposes are different: a decorator adds behavior to a class, while a proxy controls access to it. You might say, "Isn't the loading message adding behavior?" In some

ways it is; however, more importantly, the ImageProxy is controlling access to an Imagemcon. How does it control access? Well, think about it this way: the proxy is decoupling the client from the Imagemcon. If they were coupled the client would have to wait until each image is retrieved before it could paint it entire interface. The proxy controls access to the Imagemcon so that before it is fully created, the proxy provides another on screen representation. Once the Imagemcon is created the proxy allows access.

Q: How do I make clients use the Proxy rather than the Real Subject?

A: Good question. One common technique is to provide a factory that instantiates and returns the subject. Because this happens in a factory method we can then wrap the subject with a proxy before returning it. The client never knows or cares that it's using a proxy instead of the real thing.

Q: I noticed in the ImageProxy example, you always create a new Imagemcon to get the image, even if the image has already been retrieved. Could you implement something similar to the ImageProxy that caches past retrievals?

A: You are talking about a specialized form of a Virtual Proxy called a Caching Proxy. A caching proxy maintains a cache of previous created objects and when a request is made it returns cached object, if possible.

We're going to look this and at several other variants of the Proxy Pattern at the end of the chapter.

Q: I see how Decorator and Proxy relate, but what about Adapter? An adapter seems very similar as well.

A: Both Proxy and Adapter sit in front of other objects and forward requests to them. Remember that Adapter changes the interface of the objects it adapts, while the Proxy implements the same interface.

There is one additional similarity that relates to the Protection Proxy. A Protection Proxy may allow or disallow a client access to particular methods in an object based on the role of the client. In this way a Protection Proxy may only provide a partial interface to a client, which is quite similar to some Adapters. We are going to take a look at Protection Proxy in a few pages.

Fireside Chats



Tonight's talk: **Proxy and Decorator get intentional.**

Proxy

Hello, Decorator. I presume you're here because people sometimes get us confused?

Me copying *your* ideas? Please. I control access to objects. You just decorate them. My job is so much more important than yours it's just not even funny.

Fine, so maybe you're not entirely frivolous... but I still don't get why you think I'm copying all your ideas. I'm all about representing my subjects, not decorating them.

I don't think you get it, Decorator. I stand in for my Subjects; I don't just add behavior. Clients use me as a surrogate of a Real Subject, because I can protect them from unwanted access, or keep their GUIs from hanging up while they're waiting for big objects to load, or hide the fact that their Subjects are running on remote machines. I'd say that's a very different intent from yours!

Decorator

Well, I think the reason people get us confused is that you go around pretending to be an entirely different pattern, when in fact, you're just a Decorator in disguise. I really don't think you should be copying all my ideas.

"Just" decorate? You think decorating is some frivolous unimportant pattern? Let me tell you buddy, I add *behavior*. That's the most important thing about objects - what they *do*!

You can call it "representation" but if it looks like a duck and walks like a duck... I mean, just look at your Virtual Proxy; it's just another way of adding behavior to do something while some big expensive object is loading, and your Remote Proxy is a way of talking to remote objects so your clients don't have to bother with that themselves. It's all about behavior, just like I said.

Call it what you want. I implement the same interface as the objects I wrap; so do you.

Proxy

Okay, let's review that statement. You wrap an object. While sometimes we informally say a proxy wraps its Subject, that's not really an accurate term.

Think about a remote proxy... what object am I wrapping? The object I'm representing and controlling access to lives on another machine! Let's see you do that.

Sure, okay, take a virtual proxy... think about the CD viewer example. When the client first uses me as a proxy the subject doesn't even exist! So what am I wrapping there?

I never knew decorators were so dumb! Of course I sometimes create objects, how do you think a virtual proxy gets its subject! Okay, you just pointed out a big difference between us: we both know decorators only add window dressing; they never get to instantiate anything.

Hey, after this conversation I'm convinced you're just a dumb proxy!

Very seldom will you ever see a proxy get into wrapping a subject multiple times; in fact, if you're wrapping something 10 times, you better go back reexamine your design.

Decorator

Oh yeah? Why not?

Okay, but we all know remote proxies are kinda weird. Got a second example? I doubt it.

Uh huh, and the next thing you'll be saying is that you actually get to create objects.

Oh yeah? Instantiate this!

Dumb proxy? I'd like to see you recursively wrap an object with 10 decorators and keep your head straight at the same time.

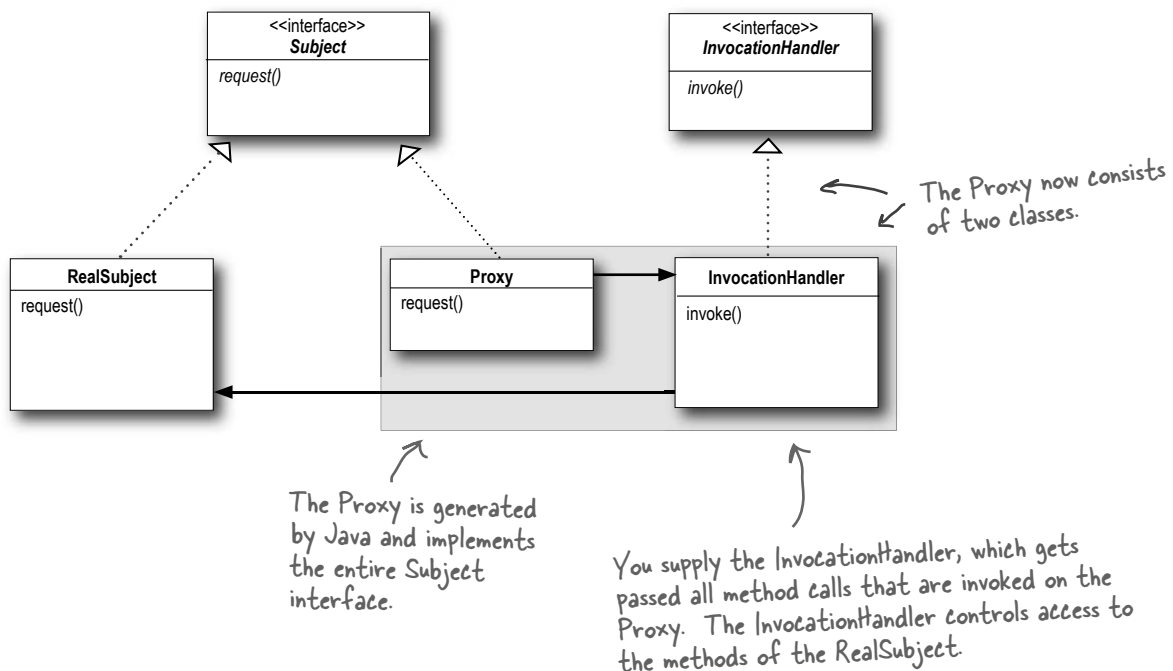
Just like a proxy, acting all real when in fact you just stand in for the objects doing the real work. You know, I actually feel sorry for you.

Using the Java API's Proxy to create a protection proxy



Java's got its own proxy support right in the `java.lang.reflect` package. With this package, Java lets you create a proxy class *on the fly* that implements one or more interfaces and forwards method invocations to a class that you specify. Because the actual proxy class is created at runtime, we refer to this Java technology as a *dynamic proxy*.

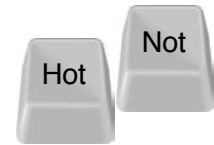
We're going to use Java's dynamic proxy to create our next proxy implementation (a protection proxy), but before we do that, let's quickly look at a class diagram that shows how dynamic proxies are put together. Like most things in the real world, it differs slightly from the classic definition of the pattern:



Because Java creates the Proxy class *for you*, you need a way to tell the Proxy class what to do. You can't put that code into the Proxy class like we did before, because you're not implementing one directly. So, if you can't put this code in the Proxy class, where do you put it? In an InvocationHandler. The job of the InvocationHandler is to respond to any method calls on the proxy. Think of the InvocationHandler as the object the Proxy asks to do all the real work after it's received the method calls.

Okay, let's step through how to use the dynamic proxy...

Matchmaking in Objectville



Every town needs a matchmaking service, right? You've risen to the task and implemented a dating service for Objectville. You've also tried to be innovative by including a "Hot or Not" feature in the service where participants can rate each other – you figure this keeps your customers engaged and looking through possible matches; it also makes things a lot more fun.

Your service revolves around a Person bean that allows you to set and get information about a person:

This is the interface; we'll get to the implementation in just a sec...

```
public interface PersonBean {

    String getName();
    String getGender();
    String getInterests();
    int getHotOrNotRating();

    void setName(String name);
    void setGender(String gender);
    void setInterests(String interests);
    void setHotOrNotRating(int rating);

}
```

Here we can get information about the person's name, gender, interests and HotOrNot rating (1-10).

setHotOrNotRating() takes an integer and adds it to the running average for this person.

We can also set the same information through the respective method calls.

Now let's check out the implementation...

The PersonBean implementation

The PersonBeanImpl implements the PersonBean interface

```
public class PersonBeanImpl implements PersonBean {
```

```
    String name;  
    String gender;  
    String interests;  
    int rating;  
    int ratingCount = 0;
```

← The instance variables.

```
    public String getName() {  
        return name;  
    }
```

```
    public String getGender() {  
        return gender;  
    }
```

```
    public String getInterests() {  
        return interests;  
    }
```

```
    public int getHotOrNotRating() {  
        if (ratingCount == 0) return 0;  
        return (rating/ratingCount);  
    }
```

← All the getter methods; they each return the appropriate instance variable...

← ...except for getHotOrNotRating(), which computes the average of the ratings by dividing the ratings by the ratingCount.

```
    public void setName(String name) {  
        this.name = name;  
    }
```

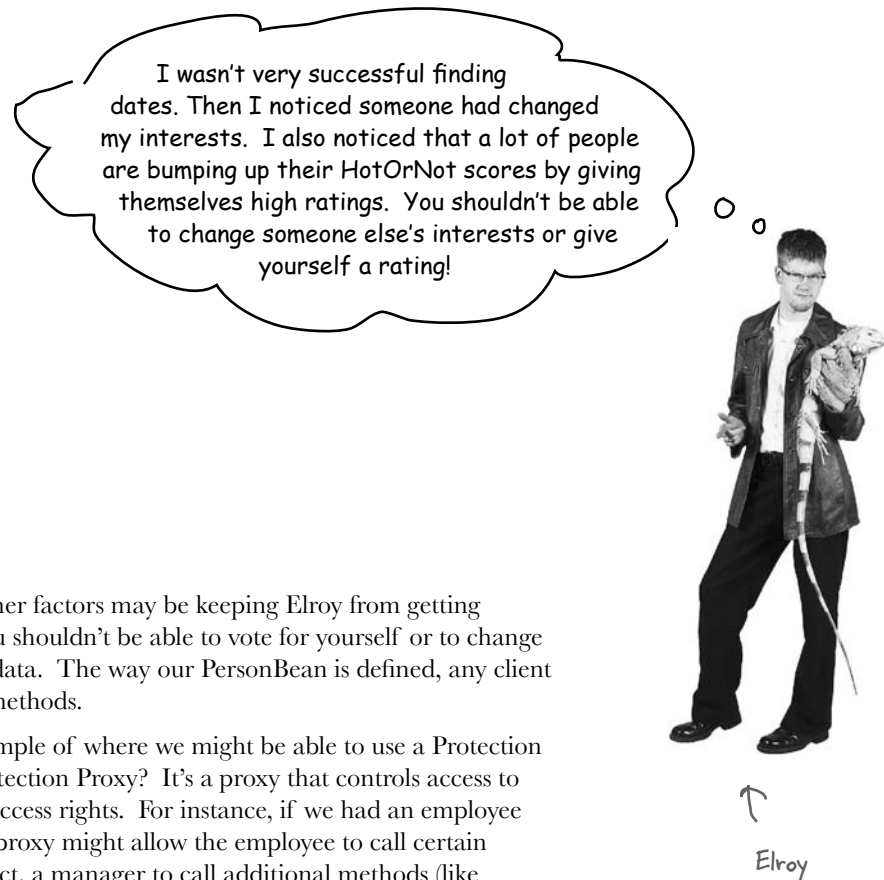
```
    public void setGender(String gender) {  
        this.gender = gender;  
    }
```

```
    public void setInterests(String interests) {  
        this.interests = interests;  
    }
```

```
    public void setHotOrNotRating(int rating) {  
        this.rating += rating;  
        ratingCount++;  
    }
```

← And here's all the setter methods, which set the corresponding instance variable.

← Finally, the setHotOrNotRating() method increments the total ratingCount and adds the rating to the running total.



While we suspect other factors may be keeping Elroy from getting dates, he is right: you shouldn't be able to vote for yourself or to change another customer's data. The way our `PersonBean` is defined, any client can call any of the methods.

This is a perfect example of where we might be able to use a Protection Proxy. What's a Protection Proxy? It's a proxy that controls access to an object based on access rights. For instance, if we had an employee object, a protection proxy might allow the employee to call certain methods on the object, a manager to call additional methods (like `setSalary()`), and a human resources employee to call any method on the object.

In our dating service we want to make sure that a customer can set his own information while preventing others from altering it. We also want to allow just the opposite with the `HotOrNot` ratings: we want the other customers to be able to set the rating, but not that particular customer. We also have a number of getter methods in the `PersonBean`, and because none of these return private information, any customer should be able to call them.



Five minute drama: protecting subjects

The Internet bubble seems a distant memory; those were the days when all you needed to do to find a better, higher-paying job was to walk across the street. Even agents for software developers were in vogue...



Big Picture: creating a Dynamic Proxy for the PersonBean

We have a couple of problems to fix: customers shouldn't be changing their own HotOrNot rating and customers shouldn't be able to change other customers' personal information. To fix these problems we're going to create two proxies: one for accessing your own PersonBean object and one for accessing another customer's PersonBean object. That way, the proxies can control what requests can be made in each circumstance.

To create these proxies we're going to use the Java API's dynamic proxy that you saw a few pages back. Java will create two proxies for us; all we need to do is supply the handlers that know what to do when a method is invoked on the proxy.

Step one:

Create two **InvocationHandlers**.

InvocationHandlers implement the behavior of the proxy. As you'll see Java will take care of creating the actual proxy class and object, we just need to supply a handler that knows what to do when a method is called on it.

Step two:

Write the code that creates the dynamic proxies.

We need to write a little bit of code to generate the proxy class and instantiate it. We'll step through this code in just a bit.

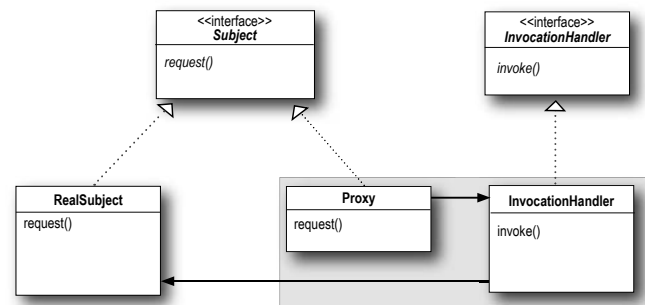
Step three:

Wrap any PersonBean object with the appropriate proxy.

When we need to use a PersonBean object, either it's the object of the customer himself (in that case, will call him the "owner"), or it's another user of the service that the customer is checking out (in that case we'll call him "non-owner").

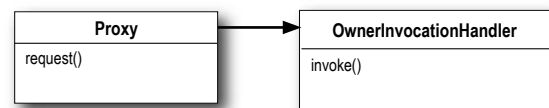
In either case, we create the appropriate proxy for the PersonBean.

Remember this diagram from a few pages back...



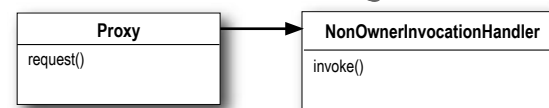
We need two of these.

We create the proxy itself at runtime.



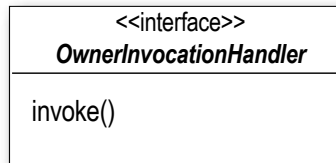
When a customer is viewing his own bean

When a customer is viewing someone else's bean



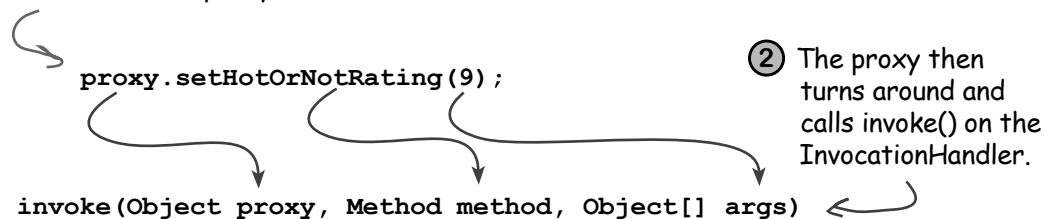
Step one: creating Invocation Handlers

We know we need to write two invocation handlers, one for the owner and one for the non-owner. But what are invocation handlers? Here's the way to think about them: when a method call is made on the proxy, the proxy forwards that call to your invocation handler, but *not* by calling the invocation handler's corresponding method. So, what does it call? Have a look at the `InvocationHandler` interface:

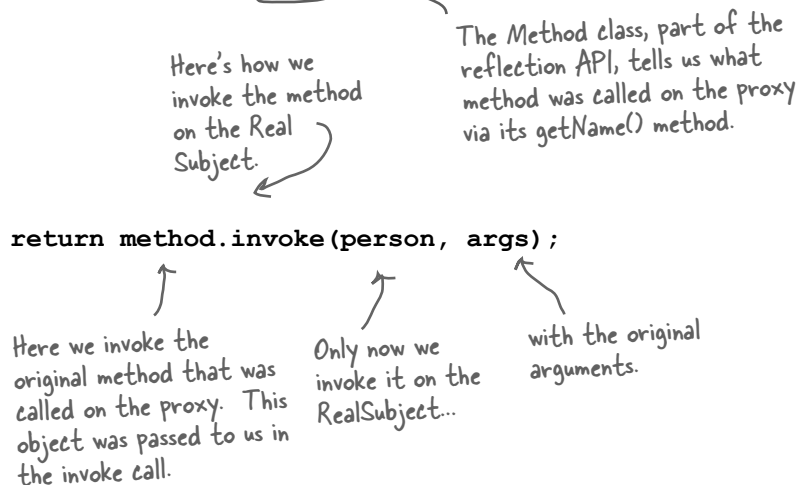


There's only one method, `invoke()`, and no matter what methods get called on the proxy, the `invoke()` method is what gets called on the handler. Let's see how this works:

- ① Let's say the `setHotOrNotRating()` method is called on the proxy.



- ③ The handler decides what it should do with the request and possibly forwards it on to the `RealSubject`. How does the handler decide? We'll find out next.



Creating Invocation Handlers continued...

When `invoke()` is called by the proxy, how do you know what to do with the call? Typically, you'll examine the method that was called on the proxy and make decisions based on the method's name and possibly its arguments. Let's implement the `OwnerInvocationHandler` to see how this works:

```
import java.lang.reflect.*;

public class OwnerInvocationHandler implements InvocationHandler {
    PersonBean person;

    public OwnerInvocationHandler(PersonBean person) {
        this.person = person;
    }

    public Object invoke(Object proxy, Method method, Object[] args)
        throws IllegalAccessException {

        try {
            if (method.getName().startsWith("get")) {
                return method.invoke(person, args);
            } else if (method.getName().equals("setHotOrNotRating")) {
                throw new IllegalAccessException();
            } else if (method.getName().startsWith("set")) {
                return method.invoke(person, args);
            }
        } catch (InvocationTargetException e) {
            e.printStackTrace();
        }
        return null;
    }
}
```

InvocationHandler is part of the `java.lang.reflect` package, so we need to import it.

All invocation handlers implement the `InvocationHandler` interface.

We're passed the Real Subject in the constructor and we keep a reference to it.

Here's the `invoke` method that gets called every time a method is invoked on the proxy.

If the method is a getter, we go ahead and invoke it on the real subject.

Otherwise, if it is the `setHotOrNotRating()` method we disallow it by throwing a `IllegalAccessException`.

Because we are the owner any other set method is fine and we go ahead and invoke it on the real subject.

This will happen if the real subject throws an exception.

If any other method is called, we're just going to return null rather than take a chance.



The `NonOwnerInvocationHandler` works just like the `OwnerInvocationHandler` except that it *allows* calls to `setHotOrNotRating()` and it *disallows* calls to any other set method. Go ahead and write this handler yourself:

Step two: creating the Proxy class and instantiating the Proxy object

Now, all we have left is to dynamically create the proxy class and instantiate the proxy object. Let's start by writing a method that takes a `PersonBean` and knows how to create an owner proxy for it. That is, we're going to create the kind of proxy that forwards its method calls to the `OwnerInvocationHandler`. Here's the code:

```
PersonBean getOwnerProxy(PersonBean person) {
    return (PersonBean) Proxy.newProxyInstance(
        person.getClass().getClassLoader(),
        person.getClass().getInterfaces(),
        new OwnerInvocationHandler(person));
}
```

This method takes a person object (the real subject) and returns a proxy for it. Because the proxy has the same interface as the subject, we return a `PersonBean`.

This code creates the proxy. Now this is some mighty ugly code, so let's step through it carefully.

To create a proxy we use the static `newProxyInstance` method on the `Proxy` class...

← We pass it the classloader for our subject...

...and the set of interfaces the proxy needs to implement...

...and an invocation handler, in this case our `OwnerInvocationHandler`.

We pass the real subject into the constructor of the invocation handler. If you look back two pages you'll see this is how the handler gets access to the real subject.



Sharpen your pencil

While it is a little complicated, there isn't much to creating a dynamic proxy. Why don't you write `getNonOwnerProxy()`, which returns a proxy for the `NonOwnerInvocationHandler`:

Take it further: can you write one method `getProxy()` that takes a handler and a person and returns a proxy that uses that handler?

Testing the matchmaking service

Let's give the matchmaking service a test run and see how it controls access to the setter methods based on the proxy that is used.

```

public class MatchMakingTestDrive {
    // instance variables here

    public static void main(String[] args) {
        MatchMakingTestDrive test = new MatchMakingTestDrive();
        test.drive();
    }

    public MatchMakingTestDrive() {
        initializeDatabase();
    }

    public void drive() {
        PersonBean joe = getPersonFromDatabase("Joe Javabean");
        PersonBean ownerProxy = getOwnerProxy(joe);
        System.out.println("Name is " + ownerProxy.getName());
        ownerProxy.setInterests("bowling, Go");
        System.out.println("Interests set from owner proxy");
        try {
            ownerProxy.setHotOrNotRating(10);
        } catch (Exception e) {
            System.out.println("Can't set rating from owner proxy");
        }
        System.out.println("Rating is " + ownerProxy.getHotOrNotRating());

        PersonBean nonOwnerProxy = getNonOwnerProxy(joe);
        System.out.println("Name is " + nonOwnerProxy.getName());
        try {
            nonOwnerProxy.setInterests("bowling, Go");
        } catch (Exception e) {
            System.out.println("Can't set interests from non owner proxy");
        }
        nonOwnerProxy.setHotOrNotRating(3);
        System.out.println("Rating set from non owner proxy");
        System.out.println("Rating is " + nonOwnerProxy.getHotOrNotRating());

        // other methods like getOwnerProxy and getNonOwnerProxy here
    }
}

```

Main just creates the test drive and calls its drive() method to get things going.

The constructor initializes our DB of people in the matchmaking service.

Let's retrieve a person from the DB

...and create an owner proxy.

Call a getter and then a setter

and then try to change the rating.

this shouldn't work!

Now create a non-owner proxy

...and call a getter

followed by a setter

This shouldn't work!

Then try to set the rating

This should work!

Running the code...

```
File Edit Window Help Born2BDynamic
% java MatchMakingTestDrive

Name is Joe Javabean
Interests set from owner proxy
Can't set rating from owner proxy
Rating is 7

Name is Joe Javabean
Can't set interests from non owner proxy
Rating set from non owner proxy
Rating is 5
%
```

Our Owner proxy allows getting and setting, except for the HotOrNot rating.

Our NonOwner proxy allows getting only, but also allows calls to set the HotOrNot rating.

The new rating is the average of the previous rating, 7 and the value set by the nonowner proxy, 3.

^{there are no}
Dumb Questions

Q: So what exactly is the “dynamic” aspect of dynamic proxies? Is it that I’m instantiating the proxy and setting it to a handler at runtime?

A: No, the proxy is dynamic because its class is created at runtime. Think about it: before your code runs there is no proxy class; it is created on demand from the set of interfaces you pass it.

Q: My `InvocationHandler` seems like a very strange proxy, it doesn’t implement any of the methods of the class it’s proxying.

A: That is because the `InvocationHandler` isn’t a proxy – it is a class that the proxy dispatches to for handling method calls. The proxy itself is created dynamically at runtime by the static `Proxy.newProxyInstance()` method.

Q: Is there any way to tell if a class is a Proxy class?

A: Yes. The `Proxy` class has a static method called `isProxyClass()`. Calling this method with a class will return true if the class is a dynamic proxy class. Other than that, the proxy class will act like any other class that implements a particular set of interfaces.

Q: Are there any restrictions on the types of interfaces I can pass into `newProxyInstance()`?

A: Yes, there are a few. First, it is worth pointing out that we always pass `newProxyInstance()` an array of interfaces – only interfaces are allowed, no classes. The major restrictions are that all non-public interfaces need to be from the same package. You also can’t have interfaces with clashing method names (that is, two interfaces with a method with the same signature). There are a few other minor nuances as well, so at some point you should take a look at the fine print on dynamic proxies in the javadoc.

Q: Why are you using skeletons? I thought we got rid of those back in Java 1.2.

A: You’re right; we don’t need to actually generate skeletons. As of Java 1.2, the RMI runtime can dispatch the client calls directly to the remote service using reflection. But we like to show the skeleton, because conceptually it helps you to understand that there is something under the covers that’s making that communication between the client stub and the remote service happen.

Q: I heard that in Java 5, I don’t even need to generate stubs anymore either. Is that true?

A: It sure is. In Java 5, RMI and Dynamic Proxy got together and now stubs are generated dynamically using Dynamic Proxy. The remote object’s stub is a `java.lang.reflect.Proxy` instance (with an invocation handler) that is automatically generated to handle all the details of getting the local method calls by the client to the remote object. So, now you don’t have to use `rmic` at all; everything you need to get a client talking to a remote object is handled for you behind the scenes.



WHO DOES WHAT?

Match each pattern with its description:

Pattern	Description
Decorator	Wraps another object and provides a different interface to it
Facade	Wraps another object and provides additional behavior for it
Proxy	Wraps another object to control access to it
Adapter	Wraps a bunch of objects to simplify their interface

The Proxy Zoo

Welcome to the Objectville Zoo!

You now know about the remote, virtual and protection proxies, but out in the wild you're going to see lots of mutations of this pattern. Over here in the Proxy corner of the zoo we've got a nice collection of wild proxy patterns that we've captured for your study.

Our job isn't done; we are sure you're going to see more variations of this pattern in the real world, so give us a hand in cataloging more proxies. Let's take a look at the existing collection:



Firewall Proxy
controls access to a set of network resources, protecting the subject from "bad" clients.

Habitat: often seen in the location of corporate firewall systems.

Help find a habitat

Smart Reference Proxy
provides additional actions whenever a subject is referenced, such as counting the number of references to an object.



Caching Proxy provides temporary storage for results of operations that are expensive. It can also allow multiple clients to share the results to reduce computation or network latency.

Habitat: often seen in web server proxies as well as content management and publishing systems.

Synchronization Proxy
provides safe access to
a subject from multiple
threads.



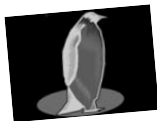
Seen hanging around JavaSpaces, where
it controls synchronized access to
an underlying set of objects in a
distributed environment.

Help find a habitat

Complexity Hiding Proxy

hides the complexity of
and controls access to a
complex set of classes.
This is sometimes called
the Facade Proxy for obvious reasons.

The Complexity Hiding Proxy differs
from the Facade Pattern in that the
proxy controls access, while the Facade
Pattern just provides an alternative
interface.



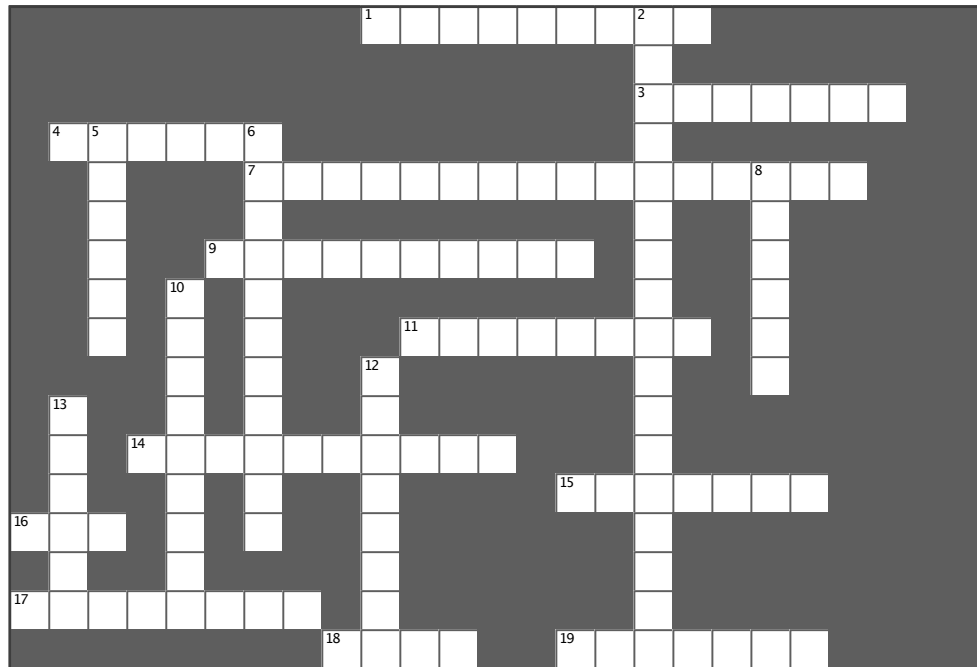
Copy-On-Write Proxy
controls the copying of
an object by deferring
the copying of an
object until it is required by
a client. This is a variant of
the Virtual Proxy.

Habitat: seen in the vicinity of the
Java 5's CopyOnWriteArrayList.

Field Notes: please add your observations of other proxies in the wild here:



It's been a LONG chapter. Why not unwind by doing a crossword puzzle before it ends?



Across

1. Group of first CD cover displayed (two words)
3. Proxy that stands in for expensive objects
4. We took one of these to learn RMI
7. Remote _____ was used to implement the gumball machine monitor (two words)
9. Software developer agent was being this kind of proxy
11. In RMI, the object that takes the network requests on the service side
14. Proxy that protects method calls from unauthorized callers
15. A _____ proxy class is created at runtime
16. Place to learn about the many proxy variants
17. Commonly used proxy for web services (two words)
18. In RMI, the proxy is called this
19. The CD viewer used this kind of proxy

Down

2. Java's dynamic proxy forwards all requests to this (two words)
5. Group that did the album MCMXC A.D.
6. This utility acts as a lookup service for RMI
8. Why Elroy couldn't get dates
10. Similar to proxy, but with a different purpose
12. Objectville Matchmaking gimmick (three words)
13. Our first mistake: the gumball machine reporting was not _____



Tools for your Design Toolbox

Your design toolbox is almost full; you're prepared for almost any design problem that comes your way.

OO Principles

Encapsulate what varies.
 Favor composition over inheritance.
 Program to interfaces, not implementations.
 Strive for loosely coupled designs between objects that interact.
 Classes should be open for extension but closed for modification.
 Depend on abstractions. Do not depend on concrete classes.
 Only talk to your friends.
 Don't call us, we'll call you.
 A class should have only one reason to change.

Abstraction
 Encapsulation
 Polymorphism
 Inheritance

No new principles this chapter, can you close the book and remember them all?

OO Patterns

Singleton
 Factory
 Abstract Factory
 Builder
 Prototype
 Strategy
 Command
 Observer
 Iterator
 Visitor
 Mediator
 Proxy

Our new pattern.
 A Proxy acts as a representative for another object.

Proxy - Provide a surrogate or placeholder for another object to control access to it.



BULLET POINTS

- The Proxy Pattern provides a representative for another object in order to control the client's access to it. There are a number of ways it can manage that access.
- A Remote Proxy manages interaction between a client and a remote object.
- A Virtual Proxy controls access to an object that is expensive to instantiate.
- A Protection Proxy controls access to the methods of an object based on the caller.
- Many other variants of the Proxy Pattern exist including caching proxies, synchronization proxies, firewall proxies, copy-on-write proxies, and so on.
- Proxy is structurally similar to Decorator, but the two differ in their purpose.
- The Decorator Pattern adds behavior to an object, while a Proxy controls access.
- Java's built-in support for Proxy can build a dynamic proxy class on demand and dispatch all calls on it to a handler of your choosing.
- Like any wrapper, proxies will increase the number of classes and objects in your designs.



Exercise solutions



Exercise

The `NonOwnerInvocationHandler` works just like the `OwnerInvocationHandler`, except that it allows calls to `setHotOrNotRating()` and it disallows calls to any other set method. Go ahead and write this handler yourself:

```
import java.lang.reflect.*;

public class NonOwnerInvocationHandler implements InvocationHandler {
    PersonBean person;

    public NonOwnerInvocationHandler(PersonBean person) {
        this.person = person;
    }

    public Object invoke(Object proxy, Method method, Object[] args)
        throws IllegalAccessException {

        try {
            if (method.getName().startsWith("get")) {
                return method.invoke(person, args);
            } else if (method.getName().equals("setHotOrNotRating")) {
                return method.invoke(person, args);
            } else if (method.getName().startsWith("set")) {
                throw new IllegalAccessException();
            }
        } catch (InvocationTargetException e) {
            e.printStackTrace();
        }
        return null;
    }
}
```

Design Class

Our `ImageProxy` class appears to have two states that are controlled by conditional statements. Can you think of another pattern that might clean up this code? How would you redesign `ImageProxy`?

Use State Pattern: implement two states, `ImageLoaded` and `ImageNotLoaded`. Then put the code from the if statements into their respective states. Start in the `ImageNotLoaded` state and then transition to the `ImageLoaded` state once the `ImageIcon` had been retrieved.



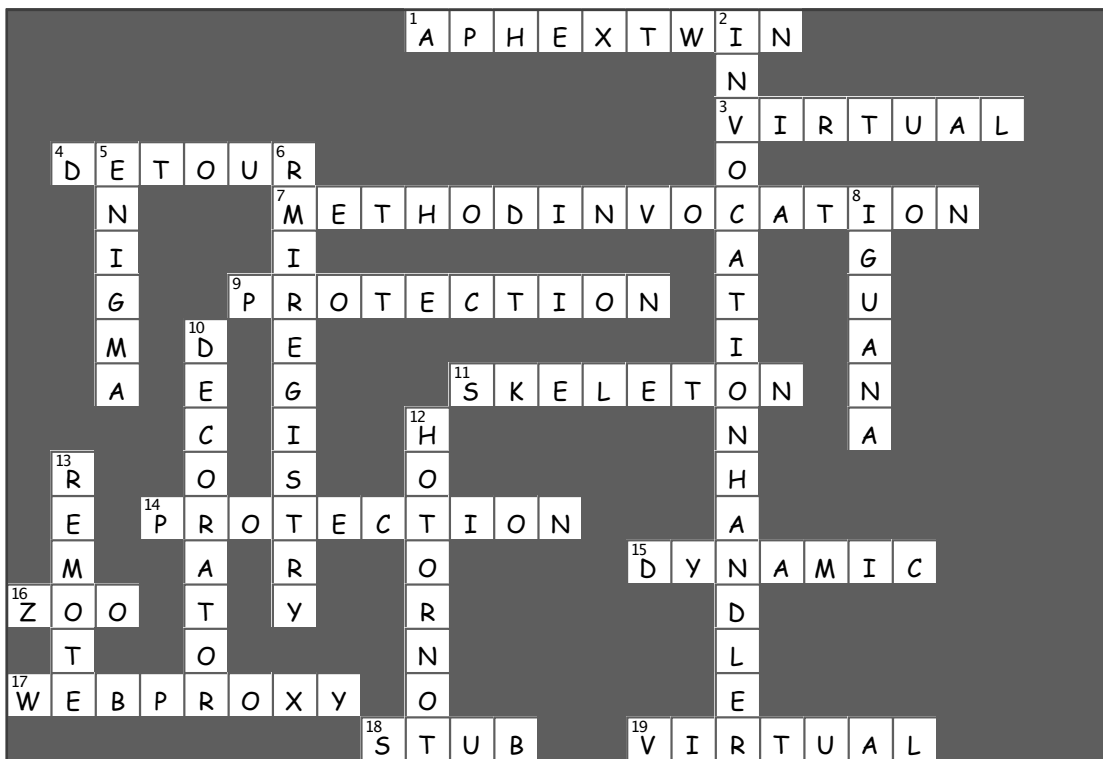
Exercise solutions



While it is a little complicated, there isn't much to creating a dynamic proxy. Why don't you write `getNonOwnerProxy()`, which returns a proxy for the `NonOwnerInvocationHandler`:

```
PersonBean getNonOwnerProxy(PersonBean person) {

    return (PersonBean) Proxy.newProxyInstance(
        person.getClass().getClassLoader(),
        person.getClass().getInterfaces(),
        new NonOwnerInvocationHandler(person));
}
```





Ready-bake Code

The code for the CD Cover Viewer

```
package headfirst.proxy.virtualproxy;
import java.net.*;
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
import java.util.*;

public class ImageProxyTestDrive {
    ImageComponent imageComponent;
    JFrame frame = new JFrame("CD Cover Viewer");
    JMenuBar menuBar;
    JMenu menu;
    Hashtable cds = new Hashtable();

    public static void main (String[] args) throws Exception {
        ImageProxyTestDrive testDrive = new ImageProxyTestDrive();
    }

    public ImageProxyTestDrive() throws Exception{
        cds.put("Ambient: Music for Airports","http://images.amazon.com/images/P/
B000003S2K.01.LZZZZZZZ.jpg");
        cds.put("Buddha Bar","http://images.amazon.com/images/P/B00009XBYK.01.LZZZZZZZ.
jpg");
        cds.put("Ima","http://images.amazon.com/images/P/B000005IRM.01.LZZZZZZZ.jpg");
        cds.put("Karma","http://images.amazon.com/images/P/B000005DCB.01.LZZZZZZZ.gif");
        cds.put("MCMXC A.D.","http://images.amazon.com/images/P/B000002URV.01.LZZZZZZZ.
jpg");
        cds.put("Northern Exposure","http://images.amazon.com/images/P/B000003SFN.01.
LZZZZZZZ.jpg");
        cds.put("Selected Ambient Works, Vol. 2","http://images.amazon.com/images/P/
B000002MNZ.01.LZZZZZZZ.jpg");
        cds.put("oliver","http://www.cs.yale.edu/homes/freeman-elisabeth/2004/9/Oliver_
sm.jpg");

        URL initialURL = new URL((String)cds.get("Selected Ambient Works, Vol. 2"));
        menuBar = new JMenuBar();
        menu = new JMenu("Favorite CDs");
        menuBar.add(menu);
        frame.setJMenuBar(menuBar);
    }
}
```

```

        for(Enumeration e = cds.keys(); e.hasMoreElements();) {
            String name = (String)e.nextElement();
            JMenuItem menuItem = new JMenuItem(name);
            menu.add(menuItem);
            menuItem.addActionListener(new ActionListener() {
                public void actionPerformed(ActionEvent event) {
                    imageComponent.setIcon(new ImageProxy(getCDUrl(event.getActionCom-
mand())));
                    frame.repaint();
                }
            });
        }

        // set up frame and menus

        Icon icon = new ImageProxy(initialURL);
        imageComponent = new ImageComponent(icon);
        frame.getContentPane().add(imageComponent);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setSize(800,600);
        frame.setVisible(true);
    }

    URL getCDUrl(String name) {
        try {
            return new URL((String)cds.get(name));
        } catch (MalformedURLException e) {
            e.printStackTrace();
            return null;
        }
    }
}

```



**Ready-bake
Code**

**The code for the CD Cover Viewer,
continued...**

```
package headfirst.proxy.virtualproxy;
import java.net.*;
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

class ImageProxy implements Icon {
    ImageIcon imageIcon;
    URL imageURL;
    Thread retrievalThread;
    boolean retrieving = false;

    public ImageProxy(URL url) { imageURL = url; }

    public int getIconWidth() {
        if (imageIcon != null) {
            return imageIcon.getIconWidth();
        } else {
            return 800;
        }
    }

    public int getIconHeight() {
        if (imageIcon != null) {
            return imageIcon.getIconHeight();
        } else {
            return 600;
        }
    }

    public void paintIcon(final Component c, Graphics g, int x, int y) {
        if (imageIcon != null) {
            imageIcon.paintIcon(c, g, x, y);
        } else {
            g.drawString("Loading CD cover, please wait...", x+300, y+190);
            if (!retrieving) {
                retrieving = true;

                retrievalThread = new Thread(new Runnable() {
                    public void run() {
                        try {
                            imageIcon = new ImageIcon(imageURL, "CD Cover");
                            c.repaint();
                        } catch (Exception e) {
```

```

        e.printStackTrace();
    }
}
});
retrievalThread.start();
}
}
}
}
}

```

```

package headfirst.proxy.virtualproxy;
import java.awt.*;
import javax.swing.*;

class ImageComponent extends JComponent {
    private Icon icon;

    public ImageComponent(Icon icon) {
        this.icon = icon;
    }

    public void setIcon(Icon icon) {
        this.icon = icon;
    }

    public void paintComponent(Graphics g) {
        super.paintComponent(g);
        int w = icon.getIconWidth();
        int h = icon.getIconHeight();
        int x = (800 - w)/2;
        int y = (600 - h)/2;
        icon.paintIcon(this, g, x, y);
    }
}

```


12 Compound Patterns

Patterns of Patterns



Who would have ever guessed that Patterns could work together?

You've already witnessed the acrimonious Fireside Chats (and you haven't even seen the Pattern Death Match pages that the editor forced us to remove from the book*), so who would have thought patterns can actually get along well together? Well, believe it or not, some of the most powerful OO designs use several patterns together. Get ready to take your pattern skills to the next level; it's time for compound patterns.

* send us email for a copy.

Working together

One of the best ways to use patterns is to get them out of the house so they can interact with other patterns. The more you use patterns the more you're going to see them showing up together in your designs. We have a special name for a set of patterns that work together in a design that can be applied over many problems: a *compound pattern*. That's right, we are now talking about patterns made of patterns!

You'll find a lot of compound patterns in use in the real world. Now that you've got patterns in your brain, you'll see that they are really just patterns working together, and that makes them easier to understand.

We're going to start this chapter by revisiting our friendly ducks in the SimUDuck duck simulator. It's only fitting that the ducks should be here when we combine patterns; after all, they've been with us throughout the entire book and they've been good sports about taking part in lots of patterns. The ducks are going to help you understand how patterns can work together in the same solution. But just because we've combined some patterns doesn't mean we have a solution that qualifies as a compound pattern. For that, it has to be a general purpose solution that can be applied to many problems. So, in the second half of the chapter we'll visit a *real* compound pattern: that's right, Mr. Model-View-Controller himself. If you haven't heard of him, you will, and you'll find this compound pattern is one of the most powerful patterns in your design toolbox.



Patterns are often used together and combined within the same design solution.

A compound pattern combines two or more patterns into a solution that solves a recurring or general problem.

Duck reunion

As you've already heard, we're going to get to work with the ducks again. This time the ducks are going to show you how patterns can coexist and even cooperate within the same solution.

We're going to rebuild our duck simulator from scratch and give it some interesting capabilities by using a bunch of patterns. Okay, let's get started...

① First, we'll create a Quackable interface.

Like we said, we're starting from scratch. This time around, the Ducks are going to implement a Quackable interface. That way we'll know what things in the simulator can quack() - like Mallard Ducks, Redhead Ducks, Duck Calls, and we might even see the Rubber Duck sneak back in.

```
public interface Quackable {
    public void quack();
}
```

Quackables only need to do one thing well: Quack!

② Now, some Ducks that implement Quackable

What good is an interface without some classes to implement it? Time to create some concrete ducks (but not the "lawn art" kind, if you know what we mean).

```
public class MallardDuck implements Quackable {
    public void quack() {
        System.out.println("Quack");
    }
}
```

Your standard Mallard duck.

```
public class RedheadDuck implements Quackable {
    public void quack() {
        System.out.println("Quack");
    }
}
```

We've got to have some variation of species if we want this to be an interesting simulator.

This wouldn't be much fun if we didn't add other kinds of Ducks too.

Remember last time? We had duck calls (those things hunters use, they are definitely quackable) and rubber ducks.

```
public class DuckCall implements Quackable {  
    public void quack() {  
        System.out.println("Kwak");  
    }  
}
```

A DuckCall that quacks but doesn't sound quite like the real thing.

```
public class RubberDuck implements Quackable {  
    public void quack() {  
        System.out.println("Squeak");  
    }  
}
```

A RubberDuck that makes a squeak when it quacks.

③ Okay, we've got our ducks; now all we need is a simulator.

Let's cook up a simulator that creates a few ducks and makes sure their quackers are working...

```
public class DuckSimulator {  
    public static void main(String[] args) {  
        DuckSimulator simulator = new DuckSimulator();  
        simulator.simulate();  
    }  
}
```

Here's our main method to get everything going.

We create a simulator and then call its simulate() method.

```
void simulate() {  
    Quackable mallardDuck = new MallardDuck();  
    Quackable redheadDuck = new RedheadDuck();  
    Quackable duckCall = new DuckCall();  
    Quackable rubberDuck = new RubberDuck();  
}
```

We need some ducks, so here we create one of each Quackable...

```
System.out.println("\nDuck Simulator");
```

```
simulate(mallardDuck);  
simulate(redheadDuck);  
simulate(duckCall);  
simulate(rubberDuck);
```

... then we simulate each one.

```
void simulate(Quackable duck) {  
    duck.quack();  
}
```

Here we overload the simulate method to simulate just one duck.

Here we let polymorphism do its magic: no matter what kind of Quackable gets passed in, the simulate() method asks it to quack.